

NB4 — Medallion Pipeline (Bronze → Silver → Gold), lightweight

Use case: LLM observability — exact schema from slide §8 (Lakehouse cho AI/ML) medallion frame. Maps to deliverable bullet 4 (the Milestone-1 Lakehouse artifact).

Pre-req: ran `make data` — but if you jumped straight here, the cell below generates the Bronze sample for you rather than failing on a missing path.

```
In [1]: import _setup # noqa: F401 -- adds scripts/ to sys.path
from pathlib import Path

import polars as pl
import duckdb
from deltalake import DeltaTable, write_deltalake
from lakehouse import path, reset

BRONZE = path("bronze", "llm_calls_raw")
SILVER = path("silver", "llm_calls")
GOLD   = path("gold",   "llm_daily_metrics")

# Self-healing pre-req (same pattern as NB7/NB8). Without this, skipping
# `make data` surfaces as a raw `Os { code: 2, kind: NotFound }` from the Rust
# layer — technically correct, useless to a student.
if not Path(BRONZE).exists():
    print("Bronze not found — running scripts/generate_data_lite.py first ...")
    import generate_data_lite

    generate_data_lite.main()
```

Bronze — verify raw is loaded

```
In [2]: bronze_n = DeltaTable(BRONZE).to_pyarrow_table().num_rows
print(f"Bronze rows: {bronze_n:,}")
print(pl.from_arrow(DeltaTable(BRONZE).to_pyarrow_table().slice(0, 2)))
```

Bronze rows: 200,000

shape: (2, 3)

request_id --- str	ts --- datetime[μs, UTC]	raw_json --- str
bc4c2724-374b-4710 -b449-f52654...	2026-04-01 00:00:00 UTC	{"model": "claude -sonnet-4-6", ...
3333e19c-f165-4b3a -b51d-3af0ec...	2026-04-01 00:00:03 UTC	{"model": "claude -haiku-4-5", ...

Silver — parse, validate, dedup

Rules: drop malformed JSON, dedupe by `request_id`, project typed columns.

```
In [3]: reset(SILVER)

# DuckDB does the JSON parse + dedup in one query – Polars also works,
# DuckDB just has nicer JSON syntax for this case.
# DuckDB reads Delta through Arrow, not through `delta_scan()`. The latter
# autoLoads an extension over the network; Arrow registration is offline and
# zero-copy, so the Lab works on a Locked-down machine.
con = duckdb.connect()
con.register("bronze", DeltaTable(BRONZE).to_pyarrow_table())

silver_arrow = con.sql(f"""
    WITH parsed AS (
        SELECT
            request_id,
            ts,
            CAST(ts AS DATE) AS date,
            json_extract_string(raw_json, '$.model') AS model,
            json_extract_string(raw_json, '$.user_id') AS user_id,
            CAST(json_extract(raw_json, '$.usage.input') AS INTEGER) AS prompt_tokens,
            CAST(json_extract(raw_json, '$.usage.output') AS INTEGER) AS completion_tok
            CAST(json_extract(raw_json, '$.latency_ms') AS INTEGER) AS latency_ms,
            json_extract_string(raw_json, '$.status') AS status,
            ROW_NUMBER() OVER (PARTITION BY request_id ORDER BY ts) AS rn
        FROM bronze
    )
    SELECT request_id, ts, date, model, user_id,
           prompt_tokens, completion_tokens, latency_ms, status
    FROM parsed
    WHERE rn = 1 AND model IS NOT NULL
""").arrow()

write_deltalake(SILVER, silver_arrow, mode="overwrite", partition_by=["date"])

silver_n = DeltaTable(SILVER).to_pyarrow_table().num_rows
print(f"Silver rows: {silver_n:,} (Bronze {bronze_n:,} → dedup dropped {bronze_n -
assert silver_n < bronze_n, (
    "Silver has the same row count as Bronze – dedup did not run. "
    "Did you regenerate Bronze with the latest generator (which injects retries)?"
)
```

Silver rows: 190,052 (Bronze 200,000 → dedup dropped 9,948)

Gold — aggregate to (date, model) metrics

```
In [4]: reset(GOLD)

# Illustrative cost model – NOT canonical pricing.
# (input USD / 1M tokens, output USD / 1M tokens)
COST_TABLE = """
```

```

VALUES
    ('claude-haiku-4-5', 0.80, 4.00),
    ('claude-sonnet-4-6', 3.00, 15.00),
    ('claude-opus-4-7', 15.00, 75.00)
"""

con.register("silver", DeltaTable(SILVER).to_pyarrow_table())
gold_arrow = con.sql(f"""
    WITH cost(model, c_in, c_out) AS ({COST_TABLE})
    SELECT
        s.date,
        s.model,
        QUANTILE_CONT(s.latency_ms, 0.50) AS p50_latency_ms,
        QUANTILE_CONT(s.latency_ms, 0.95) AS p95_latency_ms,
        SUM(s.prompt_tokens)           AS total_prompt_tokens,
        SUM(s.completion_tokens)       AS total_completion_tokens,
        AVG(CASE WHEN s.status <> 'ok' THEN 1.0 ELSE 0.0 END) AS error_rate,
        (SUM(s.prompt_tokens) * c.c_in / 1e6) +
        (SUM(s.completion_tokens) * c.c_out / 1e6) AS cost_usd
    FROM silver s
    JOIN cost c USING (model)
    GROUP BY s.date, s.model, c.c_in, c.c_out
    ORDER BY s.date, s.model
""").arrow()

write_deltalake(GOLD, gold_arrow, mode="overwrite", partition_by=["date"])

# Z-order for fast filter-by-model dashboards
DeltaTable(GOLD).optimize.z_order(["model"])

```

```

Out[4]: {'numFilesAdded': 8,
        'numFilesRemoved': 8,
        'filesAdded': '{"avg":2768.75,"max":2770,"min":2766,"totalFiles":8,"totalSize":22150}',
        'filesRemoved': '{"avg":2687.875,"max":2689,"min":2687,"totalFiles":8,"totalSize":21503}',
        'partitionsOptimized': 8,
        'numBatches': 8,
        'totalConsideredFiles': 8,
        'totalFilesSkipped': 0,
        'preserveInsertionOrder': False,
        'plannerStrategy': 'zOrder',
        'preservedStableOrder': False,
        'maxBinSpanFiles': 1}

```

Verify Gold

```

In [5]: gold_df = pl.from_arrow(DeltaTable(GOLD).to_pyarrow_table())
print(gold_df)

# Slide-5 deliverable: "Gold p50/p95/cost qua ≥ 7 ngày". Make that explicit.
n_dates = gold_df.select("date").n_unique()
n_models = gold_df.select("model").n_unique()
print(
    f"\n—— Gold deliverable metrics ——\n"

```

```
f" Distinct dates: {n_dates:>3} (target ≥ 7)\n"
f" Distinct models: {n_models:>3}\n"
f" Total Gold rows: {gold_df.height:>3} (= dates × models)"
)
assert n_dates >= 7, (
  f"Gold has only {n_dates} dates – slide deliverable requires ≥ 7. "
  "Re-run `make data` (the generator spreads across 7 UTC days)."
```

shape: (24, 8)

date --- date	model --- str	p50 _latency_ms --- f64	p95 _latency_ms --- f64	total_prompt_tokens --- decimal[38,0]	total_completion_tokens --- decimal[38,0]	error_rate --- f64	cost_usd --- f64
2026-04-02	claude-haiku-4.5	565.0	1140.0	16651653	8245961	0.049327	46.305166
2026-04-02	claude-opus-4.7	3004.0	5972.0	5579305	2767949	0.052803	291.28575
2026-04-02	claude-sonnet-4.7	1388.0	2749.0	33145603	16392052	0.049926	345.317589
2026-04-05	claude-haiku-4.5	564.0	1135.0	16398214	8293085	0.051335	46.290911
2026-04-05	claude-opus-4.7	3109.0	5902.0	5237021	2687698	0.051874	280.132665
...	...	...	...	...	...	...	...
2026-04-07	claude-opus-4.7	3069.0	5967.0	5442975	2713496	0.061844	285.156825
2026-04-07	claude-sonnet-4.7	1391.0	2748.85	32464006	16377557	0.048627	343.055373
2026-04-06	claude-haiku	564.0	1137.75	16449453	8229255	0.04714	46.076582

202	-4-5	304	601	542399	27554	0.051	288.0
6-0	cla	7.0	3.2	3	67	339	1992
4-0	ude						
6	-op						
	us-						
	4-7						
202	cla	136	273	328397	16330	0.051	343.4
6-0	ude	8.0	6.1	09	199	293	72112
4-0	-so		5				
6	nne						
	t-4						
	-6						

— Gold deliverable metrics —

Distinct dates: 8 (target ≥ 7)
 Distinct models: 3
 Total Gold rows: 24 (= dates $\times$ models)

✓ Deliverable check

- [✓] All three tables exist under `_lakehouse/{bronze,silver,gold}/`
- [✓] Silver has fewer rows than Bronze (dedup worked)
- [✓] Gold spans ≥ 7 dates $\times$ 3 models (slide §8 medallion contract)
- [✓] Cost & error_rate columns populated and non-zero

In []: